DevSecOps Integration in CI/CD Pipelines

A Review of Existing Studies, Security Practices, and Research Gaps

Abstract

DevSecOps aims to include security throughout software development and delivery. In continuous integration and continuous delivery or deployment (CI/CD), this involves checking application code, dependencies, infrastructure settings, credentials, and release processes. Although many security tools and frameworks are available, they address different problems and have different limitations. This paper reviews selected studies and official guidance to explain how security is integrated into CI/CD pipelines. The discussion covers teamwork, vulnerability detection, infrastructure as code, identity and secrets management, software supply-chain security, and monitoring after deployment. Comparison tables summarize the methods, main findings, and limitations of the reviewed studies and frameworks. The figures show the main literature themes, the types of sources considered, the placement of security practices across the delivery lifecycle, and the remaining research questions. The reviewed material suggests that no single tool provides complete protection. Effective security also depends on appropriate permissions, reliable configuration, verification of released software, and timely responses to problems. SolarWinds, Codecov, and Log4Shell illustrate different weaknesses in software production and dependency management. The review identifies continuing challenges in detection coverage, tool integration, configuration changes, and incident response. It concludes that DevSecOps requires a combination of technical controls and clearly assigned responsibilities. This paper discusses existing research and does not report new experiments or propose a new technical system.

Keywords: DevSecOps; CI/CD; literature review; software security; infrastructure as code; supply-chain security; secrets management; continuous monitoring.

1. Introduction

Software teams use CI/CD pipelines to automate activities such as building, testing, and releasing applications. Continuous integration checks changes as developers combine their work. Continuous delivery keeps software ready for release, while continuous deployment automatically releases changes that meet the required checks. These practices can make delivery more consistent, but they also create security risks.

A pipeline connects several systems, including source repositories, package registries, testing environments, and cloud services. It may also have permission to publish software or change production infrastructure. If an attacker compromises a script, dependency, or credential, the effect can extend beyond one application. For this reason, security needs to cover both the software and the process used to deliver it.

DevSecOps brings security into the shared work of development and operations teams. It includes practices such as code review, vulnerability scanning, configuration checks, access control, and monitoring. It also involves deciding who should investigate findings, approve changes, and respond to incidents. Adding a tool is only one part of this process.

The studies and frameworks discussed in this paper examine different parts of DevSecOps. Some describe how teams adopt security practices. Others investigate weaknesses in infrastructure code or compare detection tools. Security frameworks provide requirements and

guidance for application testing, dependency management, and artifact verification. These contributions are useful, but they should be interpreted according to their purpose and limitations.

This review examines how existing work explains security integration in CI/CD. It compares the approaches, discusses lessons from past incidents, and identifies questions that remain open. The review addresses four research questions:

1. **RQ1:** How do existing studies describe teamwork and technical security practices in CI/CD?
2. **RQ2:** Which security approaches are used at different stages of the delivery lifecycle?
3. **RQ3:** What are the main contributions and limitations of the reviewed studies and frameworks?
4. **RQ4:** What challenges and research gaps remain in the selected literature?

The paper is organized as a review of existing material. The figures explain relationships between topics, and the tables compare studies and approaches. No new implementation or experimental results are presented.

2. Review Scope and Approach

2.1 Sources included in the review

This paper uses a focused literature review. Sources were selected for their relevance to security in CI/CD. The selection was not based on a complete systematic search, so the review does not claim to include every relevant publication.

The material consists of five academic manuscripts and seven official project or agency sources. The academic studies cover security practices in DevOps, infrastructure-as-code research, recurring weaknesses in infrastructure scripts, defects in infrastructure automation, and Kubernetes configuration checks. The official sources discuss pipeline risks, application requirements, build information, dependency management, security-practice checks, and known exploitation.

The academic manuscripts were obtained from an author’s publication archive. All five include a common coauthor, so the review has limited diversity of research perspectives. The available manuscripts may also differ from their final published versions. Sources were retrieved on 23 October 2026, but this does not mean that the review covers all research published up to that date.

2.2 How the sources were compared

The academic studies are compared by their topic, method, reported sample, main contribution, and limitations. Sample sizes describe the studies being reviewed. They are not data collected for this paper. Frameworks are compared by the security problem they address, the guidance they provide, and the questions they leave unanswered.

Three types of sources are considered:

- **Academic studies** report findings from surveys, software repositories, defect records, or other research material.
- **Standards and guidance** describe requirements, risks, and recommended security practices.
- **Incident accounts** explain how particular security failures occurred.

These sources provide different kinds of information. A survey describes participants' experiences, while a tool study examines particular files or rules. A standard explains what should be checked, but does not prove that an organization has applied those checks correctly. An incident illustrates a failure without showing how common that failure is across all organizations.

2.3 Interpreting the findings carefully

This review separates the use of a security practice from evidence that it is effective. For example, running a scanner does not prove that all vulnerabilities have been found. Similarly, generating an alert does not prove that a team has resolved the problem.

Some source versions contain differences between their numerical summaries and other reported information. The discussion therefore emphasizes findings that can be interpreted consistently. Disputed figures and broad claims about breach frequency or remediation costs are not repeated. The review presents a qualitative comparison rather than a combined performance score.

3. Review of Existing Studies

3.1 Teamwork and shared security responsibility

One practitioner study examines online material and surveys representatives from nine organizations. It describes how teams use security requirements, configuration practices, monitoring, and collaboration within DevOps. The study shows that security integration includes organizational practices as well as automated checks.

Developers, operations staff, and security specialists work on different parts of the same delivery process. Developers change application and build code. Operations staff manage infrastructure and deployment environments. Security specialists help identify risks and interpret requirements. A security finding may need input from more than one of these groups before it can be resolved.

However, shared responsibility does not mean that everyone needs unrestricted access. Teams can work together while keeping deployment permissions and administrator accounts limited. The reviewed study also discusses risks from inappropriate access and from rapid releases that overlook security work.

The study is useful for understanding reported practices, but its sample is small and some information is self-reported. It does not prove that a particular team structure causes fewer breaches. Its main value for this review is showing why collaboration and clear responsibilities matter during adoption.

3.2 Research on infrastructure as code

A historical mapping study reviews infrastructure as code (IaC), which uses configuration files and automation to manage computing resources. It groups the literature into tools and frameworks, adoption, empirical studies, and testing. The study includes 32 publications collected through 2017, with half classified as tool or framework work.

This classification helps explain the different contributions made by research papers. A paper that introduces a tool may show that an approach is possible. It may not show whether the tool remains accurate, useful, or easy to maintain in different organizations. A study of existing defects answers a different question by examining what has already gone wrong.

The mapping is historical, so its publication counts do not describe the current field. Nevertheless, it highlights the need to distinguish between introducing a method and understanding its practical use.

IaC is important to DevSecOps because automation can repeat both secure and insecure configurations. Storing configuration in reviewable files makes inspection easier. The resulting infrastructure still depends on the settings supplied, the checks performed, and the behavior of the automation tool.

3.3 Recurring weaknesses in infrastructure scripts

A study of security smells examines 15,232 infrastructure scripts from 293 open-source repositories. A security smell is a recurring pattern that may indicate a weakness. Examples include hard-coded secrets and insecure default settings. The study uses static analysis to identify such patterns.

This work shows why infrastructure scripts should be included in security reviews. An application may have secure source code while its infrastructure exposes credentials or grants unnecessary permissions. Reviewing application code alone would miss these issues.

At the same time, a smell is a warning that needs investigation. It does not prove that an attacker has exploited the system. The risk may depend on whether the script is used in production, whether a credential is valid, and what other controls are present.

Context can also affect how developers respond. A hard-coded value in a training example may not expose a live account, but someone could copy the example into a real deployment. Reporting every example as an urgent production problem could also create unnecessary work. The study supports examining findings in context instead of accepting or rejecting every warning automatically.

The findings are also limited by the infrastructure language and repositories studied. They do not establish complete coverage of Terraform, Kubernetes, or every cloud service. Similar patterns need to be examined in the environments where they are used.

3.4 Defects in infrastructure automation

Another study examines 5,110 Ansible-related defects in state reconciliation and identifies eight categories. State reconciliation is the process of comparing the desired infrastructure state with the current state and deciding which changes to make.

This study adds an important point to the review: a configuration file is only one part of infrastructure behavior. An automation engine interprets that file using variables, inventories, providers, and remote interfaces. A defect in these components can cause the actual result to differ from the intended configuration.

Static configuration checks are still useful because they can identify known problems before deployment. Observing the deployed environment provides different information about what actually happened and what changed later. Both types of checks help explain infrastructure security.

The findings apply to the setting examined in the study. They do not show that every automation tool has the same defects. The study’s use of generated inputs to discover problems also does not prove that software can safely repair infrastructure without human review.

3.5 Kubernetes configuration checks

A Kubernetes study examines 53 security configurations prescribed by the Center for Internet Security (CIS), 544 configuration files, and 20 survey respondents. It compares the support and detection behavior of five static-analysis tools. The study reports limitations in what these tools cover and detect.

Its findings should be read in relation to the versions, rules, and files used in that study. They are not permanent rankings of current products. Some numerical summaries in the available source version also require further checking. This paper therefore discusses the findings without reproducing a performance chart.

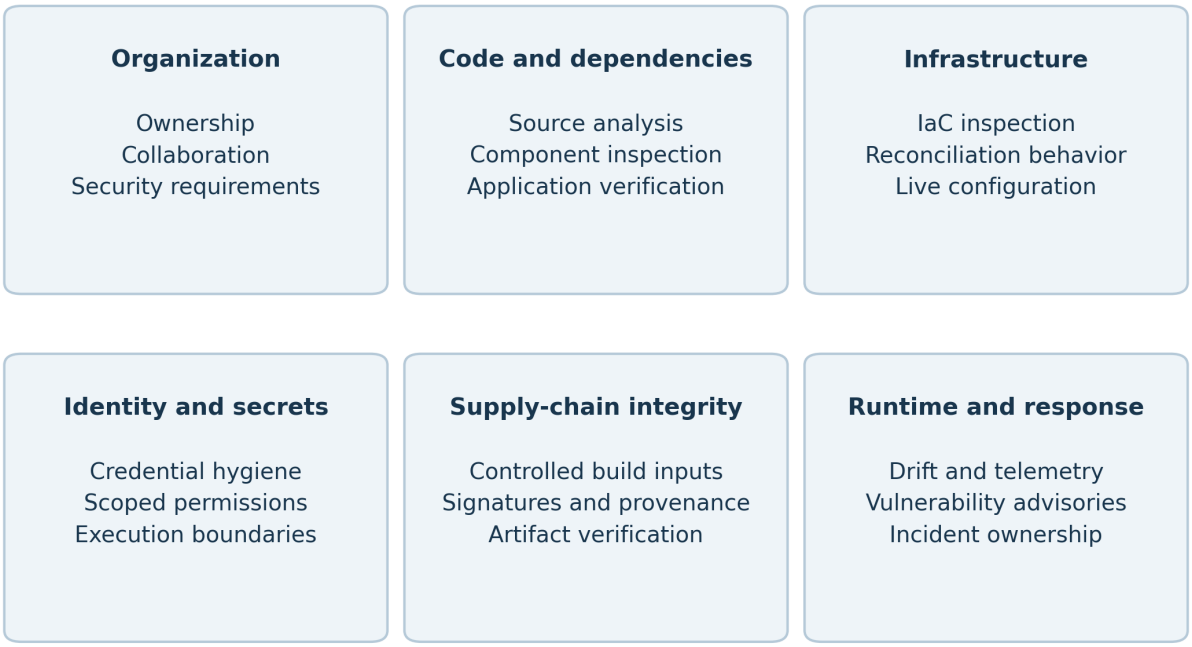
Using several tools may increase the number of patterns checked. It can also produce repeated warnings or different descriptions of the same issue. Tools with similar rules may miss the same problems. The number of alerts alone is therefore not a reliable measure of security coverage.

Configuration checks can also lack information that becomes available later. A template may be filled with values during deployment, or a file may describe only part of a cluster. The study helps explain why support for a rule, detection of a warning, and exposure in a running system are different questions.

3.6 Main themes in the literature

Figure 1 organizes the reviewed material into six themes. These cover teamwork, code and dependencies, infrastructure, identities and secrets, supply-chain integrity, and monitoring. The themes provide a simple way to follow the discussion across the studies and guidance.

Figure 1. Main topics covered by the reviewed DevSecOps literature.



Topic groups overlap; their size does not show frequency or effectiveness.

The topics overlap. For example, a dependency script may introduce unsafe code and also gain access to credentials. A configuration problem may require both a code change and a permissions change. These connections help explain why DevSecOps cannot be understood

through a list of tools alone. The figure shows topic groups, not how common or effective each practice is.

Table 1. What the reviewed sources explain and where their conclusions are limited.

Type of source	What it explains	Main limit
Practitioner research	Reported teamwork and security practices	Does not prove a general reduction in breaches
Historical literature mapping	Topics covered by a selected set of IaC papers	Does not describe the whole current research field
Security-smell research	Repeated warning patterns in infrastructure scripts	A warning does not prove successful exploitation
Automation-defect research	Problems in how infrastructure changes are carried out	An approved file does not guarantee the expected live state
Configuration-tool studies	Detection and coverage in a particular setting	Findings depend on the tools, versions, and files examined
Security standards	Requirements and recommended checks	Publication of a requirement does not prove correct use
Incident accounts	Examples of security failures	One incident does not establish overall frequency

3.7 Comparison of the reviewed studies

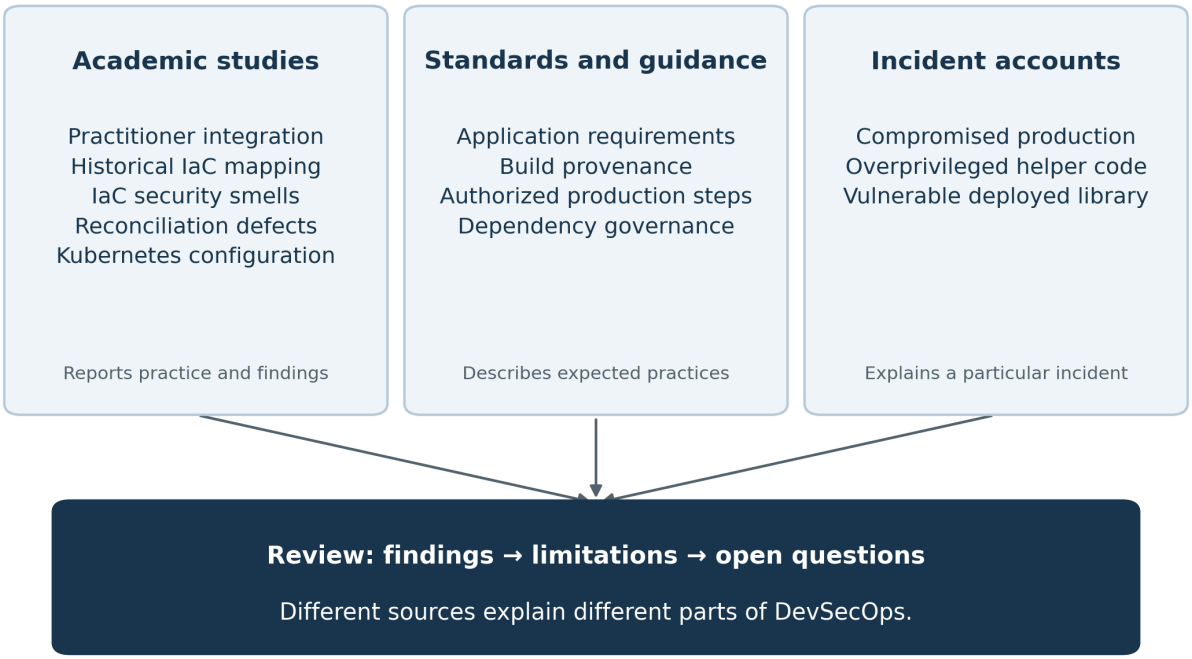
Table 2 summarizes the five academic studies. Their samples include organizations, publications, scripts, defect records, and configuration files. These are different units, so a larger sample does not automatically make one study more suitable for every question.

Table 2. Comparison of the reviewed studies, their methods, and their limitations.

Study topic	Method and reported scope	Main contribution	Main limitation
Security integration in teams	Online-material analysis and survey of nine organizations	Describes collaboration, requirements, and security practices	Small sample and partly self-reported information
Historical IaC research	Mapping of 32 publications collected through 2017	Groups work on tools, adoption, empirical studies, and testing	Historical coverage does not represent the whole current field
IaC security smells	Static analysis of 15,232 scripts from 293 repositories	Identifies recurring patterns that may indicate weaknesses	Findings depend on language and context; smells are not confirmed exploits
State reconciliation	Analysis of 5,110 Ansible-related defects and eight categories	Shows why automation behavior matters as well as configuration text	Findings are specific to the tools and setting examined
Kubernetes configuration	Five tools, 53 prescribed settings, 544 files, and 20 respondents	Identifies limitations in configuration checks and tool coverage	Version and dataset differences affect findings; some source figures need checking

Together, these studies explain different parts of security integration. They do not provide one common measure of DevSecOps effectiveness. Figure 2 shows how the review uses academic studies, standards, and incident accounts to compare existing knowledge and remaining questions.

Figure 2. Types of sources used to compare existing knowledge and open questions.



The arrows organize the review; they do not show cause and effect.

The arrows show how the discussion is organized. They do not represent a cause-and-effect model or the screening stages of a systematic review.

4. Comparison of Existing Security Approaches

4.1 Security frameworks and their purposes

The OWASP CI/CD risk taxonomy groups common pipeline risks. These include unsafe execution of pipeline code, weak access controls, exposed credentials, poor artifact verification, and limited visibility. The taxonomy helps teams decide what to examine. It does not show how often each risk occurs or whether a particular organization has addressed it successfully.

The OWASP Application Security Verification Standard (ASVS) describes requirements for assessing application security. Some can be checked automatically, while others require manual review or an understanding of the application’s design. ASVS is a requirements standard rather than a scanner. Teams still need to show how each relevant requirement has been checked.

Supply-chain Levels for Software Artifacts (SLSA) describes expectations for build integrity and information about software production. The version reviewed here is v1.1. It uses provenance, which records how an artifact was produced, together with checks against trusted production expectations. This information can help identify unexpected production steps or changes. It does not prove that the original source code is harmless.

The in-toto specification describes expected supply-chain steps and the people or systems authorized to perform them. Signed records can be checked against these rules. This makes the expected process clearer, but protection still depends on complete rules, protected verification material, and someone actually checking the records.

The OpenSSF Secure Supply Chain Consumption Framework (S2C2F) focuses on using open-source components securely. OpenSSF Scorecard checks selected security practices and provides signals that can support maintenance and dependency decisions. The Cybersecurity and Infrastructure Security Agency (CISA) catalog identifies vulnerabilities known to have been exploited. These sources serve different purposes and should be used with that distinction in mind.

Table 3. Comparison of security frameworks and information sources.

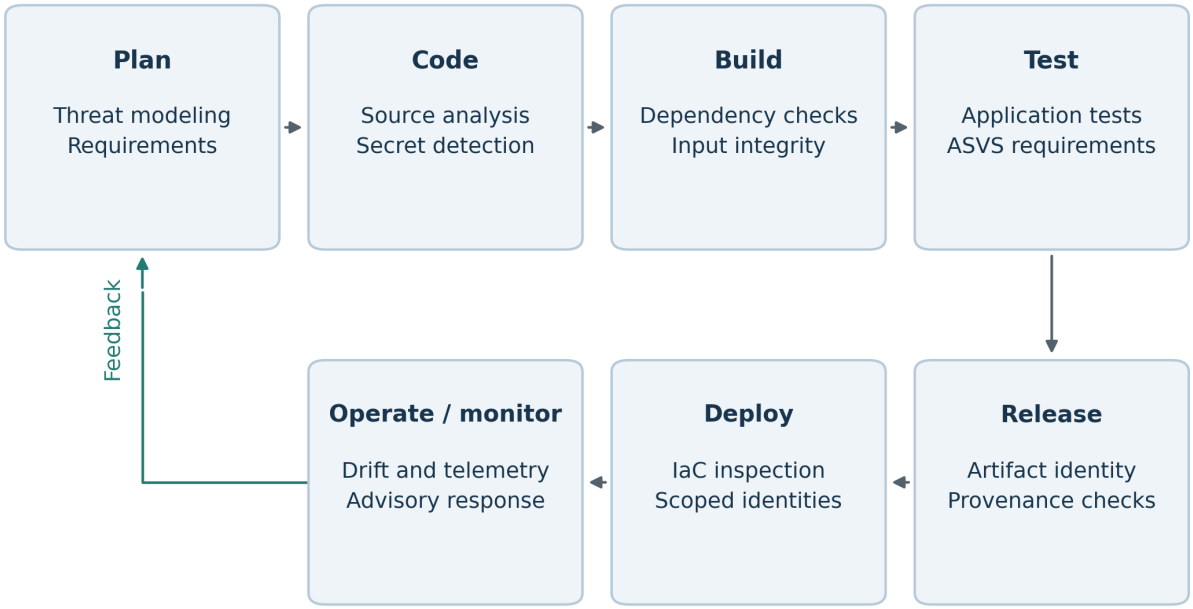
Framework or source	Main question	What it provides	Main limitation
OWASP CI/CD risk taxonomy	Which pipeline risks should be considered?	Risk categories for reviewing delivery processes	Categories do not show risk frequency or prove effective protection
OWASP ASVS 5.0.0	Which application requirements should be checked?	Requirements for automated and manual security assessment	Teams must still show which requirements were checked and how
SLSA v1.1	What is known about an artifact’s build process?	Provenance and checks against trusted build expectations	Correct build information does not prove harmless source code
in-toto 1.0.0	Were the expected steps carried out by authorized actors?	Signed process records and verification rules	Missing rules or compromised trust material can weaken the checks
OpenSSF S2C2F	How should open-source components be managed?	Guidance on dependency use and organizational practices	Guidance does not guarantee that components are free from weaknesses
OpenSSF Scorecard	Which visible security practices need attention?	Automated checks that support project and dependency review	A score does not give the probability of a compromise
CISA exploitation catalog	Is exploitation of a listed vulnerability known?	Information that helps prioritize affected software	A vulnerability may still be exploitable even if it is not listed

4.2 Security across the delivery lifecycle

Different security practices are used at different points in delivery. Planning includes threat modeling and security requirements. Coding includes source review and secret detection. Building includes dependency checks and control of build inputs. Testing examines application behavior. Release and deployment activities address artifact verification, infrastructure settings, and permissions. Monitoring continues after deployment.

Figure 3 summarizes these stages. Some practices can be repeated or used at several stages. Their placement in the figure is an example of how existing approaches fit into the lifecycle, rather than a required design for every organization.

Figure 3. Existing security practices across the software delivery lifecycle.



Existing practices; some controls can be used at more than one stage.

Information from operation and monitoring can guide later planning and maintenance. This ongoing feedback helps explain continuous security: teams need to keep reviewing conditions after a release has passed its initial checks.

4.3 Application, dependency, and infrastructure checks

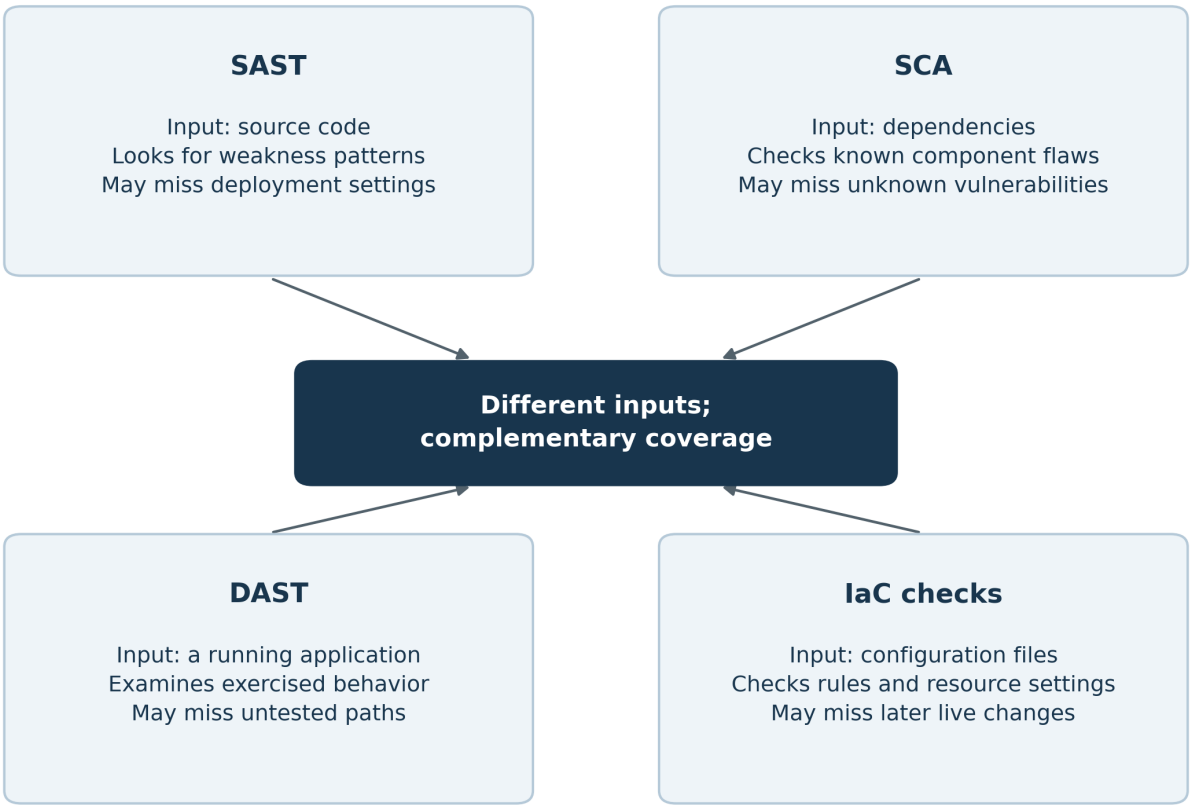
Static application security testing (SAST) examines source code or related representations for selected weakness patterns. Software composition analysis (SCA) examines dependencies and known vulnerability information. Dynamic application security testing (DAST) examines a running application’s behavior. Infrastructure checks examine settings and relationships between resources.

Each approach has limits. Source-code analysis may not reveal an unsafe cloud role assigned at deployment. A dependency check may identify an affected component without showing whether the vulnerable function is used. A configuration check may not see a later manual change. Dynamic testing may miss behavior that requires a particular account, session, or sequence of actions.

These differences explain why guidance often recommends combining approaches. However, adding more tools does not always add independent coverage. Similar rules may produce duplicate findings or miss the same weakness. A useful comparison considers what each approach can observe and what information it lacks.

Figure 4 compares the main inputs and limits of these approaches. It shows why a source-code check, a dependency check, a running-application test, and an infrastructure check should be interpreted differently. The diagram compares general methods, rather than ranking particular products.

Figure 4. Comparison of what SAST, SCA, DAST, and IaC checks examine.



General approaches, not product rankings or measured performance.

The examples describe common uses. Actual coverage depends on the tool, its configuration, and the information available to it.

4.4 Identity and secrets management

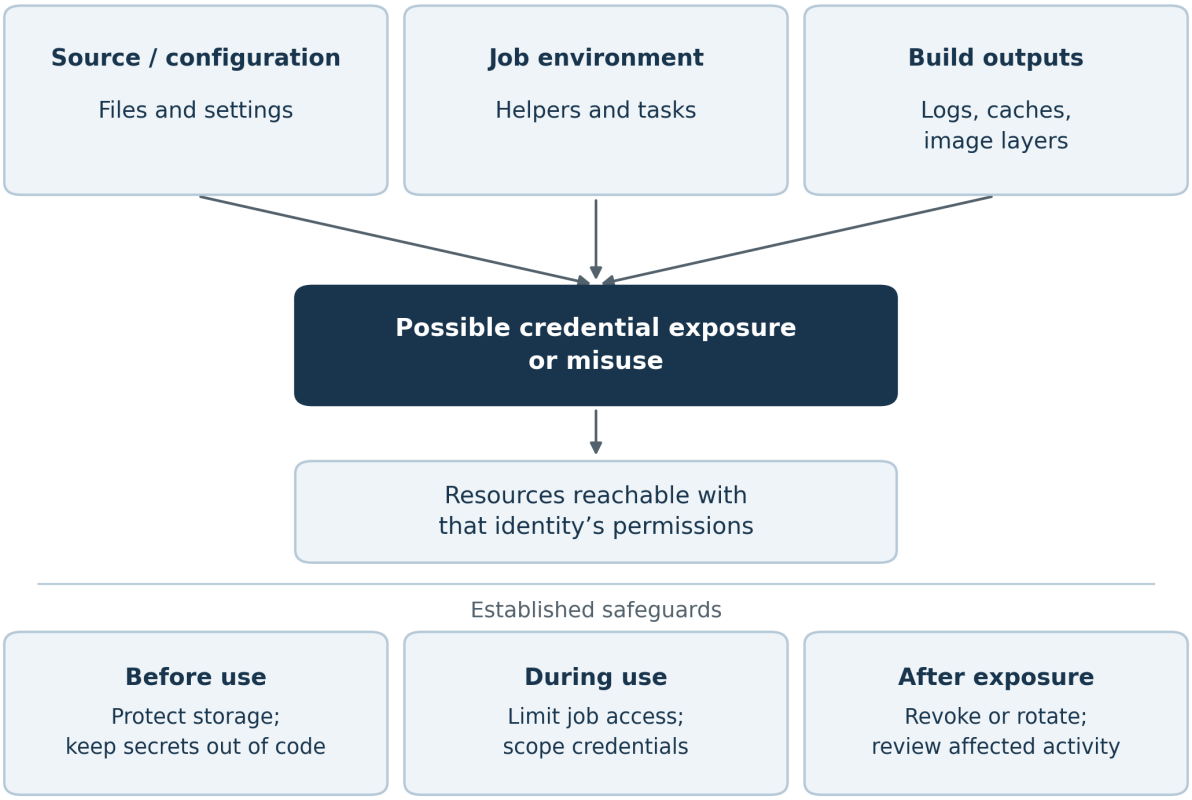
Secrets can be exposed in more places than the source repository. They may appear in logs, container layers, caches, temporary files, or build environments. A successful source scan therefore does not prove that every credential is protected.

Identity and access management (IAM) helps control what a pipeline or service is allowed to do. Existing guidance recommends limiting access according to the task being performed. Code from an unreviewed contribution should not automatically receive the permissions used to publish a release or change production infrastructure.

Temporary credentials can reduce how long an exposed credential remains useful. Their permissions still matter. A temporary administrator token can cause harm while it is valid, and broad identity rules may allow another token to be issued. Least privilege means examining the access actually granted, rather than relying on a role’s name or a token’s short lifetime.

Figure 5 links common locations of credential exposure with established safeguards. A credential’s location affects how it can be exposed, while its permissions affect which resources may be reached. The controls shown address different parts of this problem; none guarantees that credentials can never be misused.

Figure 5. Credential exposure risks and established safeguards in CI/CD.



The arrows show possible relationships, not an inevitable attack sequence. Temporary credentials can limit reuse time, but narrow permissions and an effective response are still important.

4.5 Signatures, provenance, and component inventories

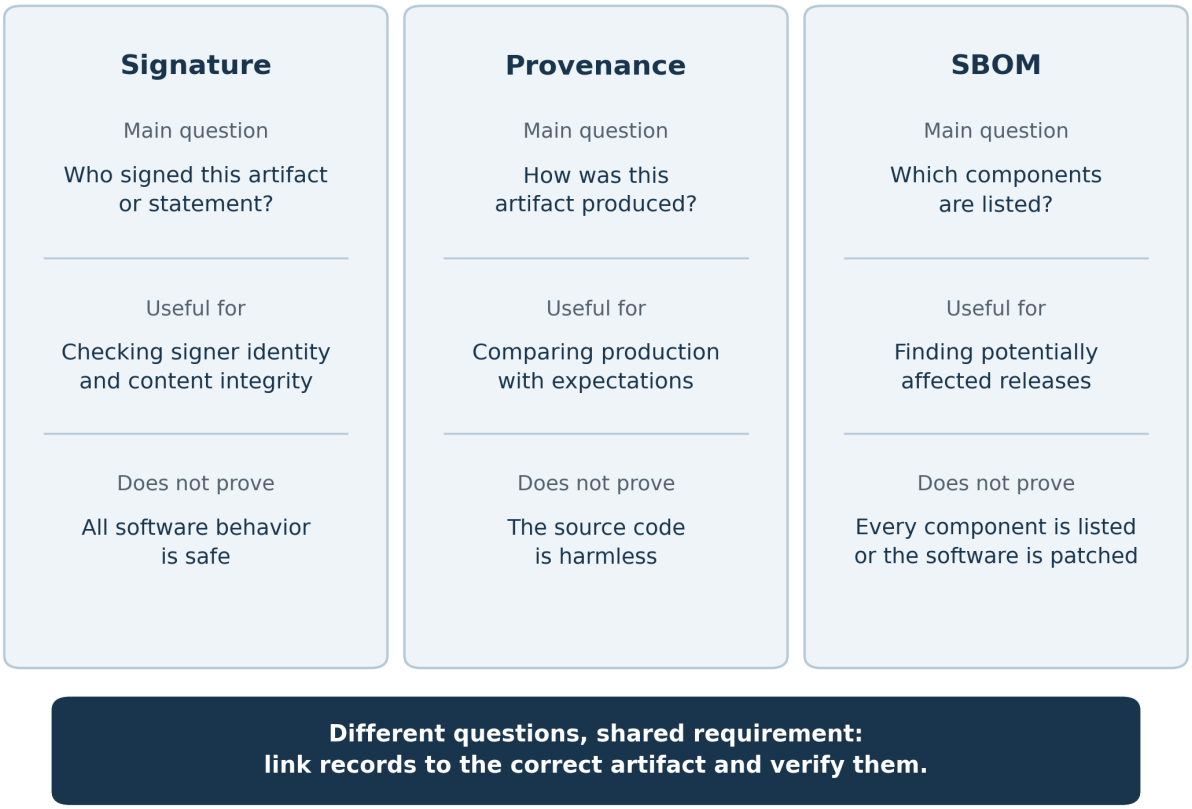
Signatures, provenance, and software inventories describe different properties. A digital signature can authenticate a statement or identify who signed an artifact. Provenance records information about how software was produced. An inventory lists its components. These records are most useful when they refer to the exact artifact being released or deployed.

An immutable digest, a hash used to identify particular file contents, can help establish that different stages handled the same bytes. A release name or changeable tag may not provide the same assurance. Verification also requires checking the signer, the expected producer, and the artifact to which a statement applies. Collecting a signed record without checking these details leaves important questions unanswered.

A software bill of materials (SBOM) can help teams locate affected releases when a new vulnerability is disclosed. It needs to be connected to deployment records and kept useful as information changes. An SBOM does not install a patch or prove that an application cannot be exploited. It supports response by helping teams understand what they use.

Figure 6 separates the questions answered by signatures, provenance, and an SBOM. These records can support one another, but they describe different properties. Their usefulness depends on connecting them to the correct artifact and checking the information before relying on it.

Figure 6. Different roles of signatures, provenance, and an SBOM.



The comparison summarizes the reviewed approaches. It does not imply that any one record proves complete software security.

5. Lessons from Past Security Incidents

The incidents in this section illustrate different weaknesses in software delivery. SolarWinds and Codecov are discussed using summaries in the reviewed guidance; their primary incident reports were not independently retrieved for this review. The Log4Shell discussion uses CISA's entry for CVE-2021-44228. These examples help explain security concerns, but they do not establish how frequently each type of failure occurs.

5.1 SolarWinds: trust in the production process

The SolarWinds incident shows why the software production process needs protection. Receiving software through a recognized supplier's release channel does not, on its own, prove that the software is harmless. Trust in a supplier, protection of the build environment, and verification of the final artifact are related but separate issues.

Existing approaches include protecting build systems, restricting changes to production scripts, checking provenance, and separating build permissions from release approval. Independent reproduction of a build can provide further information where it is practical. Even so, a malicious source change could be reproduced and signed correctly.

The main lesson is that supply-chain controls address particular risks under particular assumptions. A valid signature is useful information, but it cannot answer every question about the software's behavior or the trustworthiness of its production.

5.2 Codecov: access given to build helpers

The Codecov incident illustrates the risk of giving externally supplied helper code access to sensitive build information. A helper may perform a small task while still being able to read credentials available to the whole job. That access can create consequences beyond the helper's intended purpose.

Guidance addresses this risk through controlled inputs, verification of downloaded content, separate execution environments, limited credentials, and network restrictions. Fixing a dependency to a particular version can reduce unexpected changes. Checking it against a trusted integrity value can help detect substitution. Neither measure proves that the selected version is harmless.

This example connects dependency security with access control. Teams need to consider both where helper code comes from and what it can do when executed. Temporary credentials reduce the time available for misuse, but do not remove the need to limit permissions.

5.3 Log4Shell: responding to vulnerable dependencies

Log4Shell illustrates a different problem: a genuine and widely used component can contain a serious vulnerability. CISA identifies CVE-2021-44228 in Apache Log4j2 as a known exploited remote-code-execution vulnerability. The issue concerns vulnerable behavior in a legitimate dependency, rather than simply a substituted artifact.

The case highlights the value of knowing which components are used and where they are deployed. Inventories and deployment records can help teams identify potentially affected systems. Teams must also be able to update or contain those systems and assign responsibility for the work.

Configuration and the use of vulnerable functions can help determine exposure. However, an incomplete assessment should not be treated as proof that exploitation is impossible. Dependency security needs continuing attention because new information can change the assessment of software that has already been released.

Table 4. Comparison of past incidents and their security lessons.

Incident	Problem illustrated	Relevant security practices	Main lesson
SolarWinds	Compromise of the software production process	Protected builds, controlled changes, provenance checks, separate release approval	A trusted release channel or valid signature does not prove harmless content
Codecov	Helper code can access sensitive build information	Controlled inputs, isolated execution, limited permissions, integrity checks	The source of helper code and its execution permissions both matter
Log4Shell	A vulnerable genuine component remains in deployed software	Component inventories, advisory monitoring, updates, containment	Finding affected software is useful only when it leads to an effective response

5.4 Weak practices and better alternatives

Security problems often arise when a practice is used without considering its assumptions or the access it grants. Table 5 summarizes practices discussed in the reviewed guidance and compares them with established alternatives. The table does not estimate how often organizations use these practices.

Table 5. Comparison of weak security practices and established alternatives.

Practice	Why it can cause problems	Established alternative
Check security only before the final release	Important problems may be found after other work depends on them	Provide earlier feedback and continue checks during release and operation
Share one deployment identity across environments	A compromise can affect several projects or environments	Limit identities and permissions to their tasks and environments
Run changeable external scripts in privileged jobs	Downloaded code can change without a reviewed repository change	Control inputs, check trusted integrity values, and separate execution
Treat every warning as equally urgent	Missing context and repeated alerts can make prioritization difficult	Review relevance, assign owners, and document exceptions
Assume a signature proves software is safe	A signature authenticates a statement or origin, not all behavior	Check artifact identity and production information alongside application security
Create an SBOM only for an audit	An unused or outdated inventory does not support a timely response	Connect component information to deployments and vulnerability handling
Assume automation always restores secure infrastructure	Automation can repeatedly apply an unsafe configuration	Review the intended settings and observe the actual environment
Treat an alert as a completed response	The problem may remain unresolved without an owner or action	Maintain clear responsibilities and practical response processes

6. Common Challenges and Research Gaps

6.1 Detection coverage and context

The reviewed studies show that automated checks can identify useful warning patterns. Their coverage still depends on supported languages, rules, input files, and deployment information. A tool may understand one part of a system while missing settings added later.

An important question is how tools can communicate the assumptions behind a finding. A warning that identifies the affected resource and explains its possible consequence can be easier to investigate than a general label. This connects the accuracy of detection with the practical work of understanding and fixing a problem.

6.2 Intended configuration and actual infrastructure

Infrastructure security involves the configuration that a team intends to apply, the actions performed by automation, and the state of the running environment. These can differ because of defects, defaults, manual changes, or interactions between services.

Static checks examine the declared settings. Research on reconciliation examines how automation carries out changes. Live-state monitoring helps identify later differences. The reviewed material suggests that these sources of information are useful together, but one should not be assumed to replace all the others.

The remaining challenge is keeping these views connected, especially when several repositories, templates, and service providers are involved. This is a wider issue than checking one configuration file.

6.3 Understanding security records and trust

Supply-chain specifications make production steps and artifact relationships easier to record and verify. Their usefulness depends on who creates the records, which identities are trusted, and what checks the receiving system performs.

If the same compromised authority can create and approve a record, both activities may be affected. This does not make signed information worthless. It means that teams need to understand what the record proves and which assumptions it relies on.

Build information, application tests, and observations from a running system answer different questions. Combining them requires careful interpretation. A single score can hide differences in coverage and trust if those details are not retained.

6.4 Tool maintenance and developer adoption

Security controls need continuing maintenance. Rules and dependencies change, exceptions expire, and teams take on new responsibilities. A tool that worked during initial installation may later become inaccurate, difficult to use, or poorly matched to the system.

The practitioner study also highlights the value of collaboration and activities that cannot be fully automated. Findings need clear explanations and an owner who can act on them. Several overlapping tools can create extra work without resolving the most important risk.

The research gap concerns sustained use as well as initial adoption. More accounts from different organizations would help explain how teams maintain these practices over time.

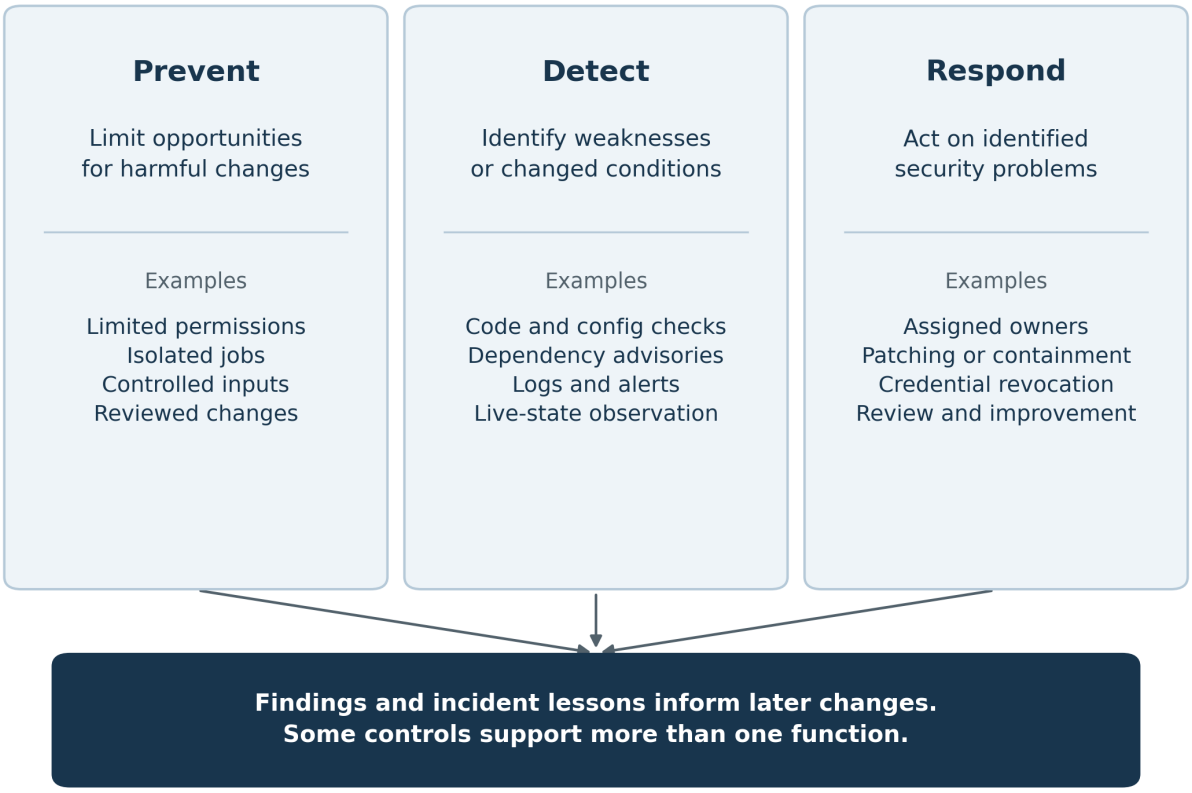
6.5 Monitoring and incident response

Monitoring can reveal changes that earlier checks cannot observe. However, logs may be incomplete, alerts may lack context, and action may be delayed when responsibilities are unclear. Collecting information is therefore only part of operational security.

Teams also need to review software after deployment. A newly disclosed vulnerability or a changed configuration can affect a release that previously passed its checks. Continuous security involves responding to this changing information rather than assuming that a successful build check remains sufficient indefinitely.

Figure 7 groups the reviewed practices by three functions: prevention, detection, and response. This distinction helps explain why finding a weakness is different from limiting the opportunity for misuse or taking action after a problem is identified. The functions overlap, and some controls can support more than one of them.

Figure 7. Prevention, detection, and response in the reviewed security practices.



This is an explanatory grouping of existing practices, not a new security framework or a measured comparison of effectiveness.

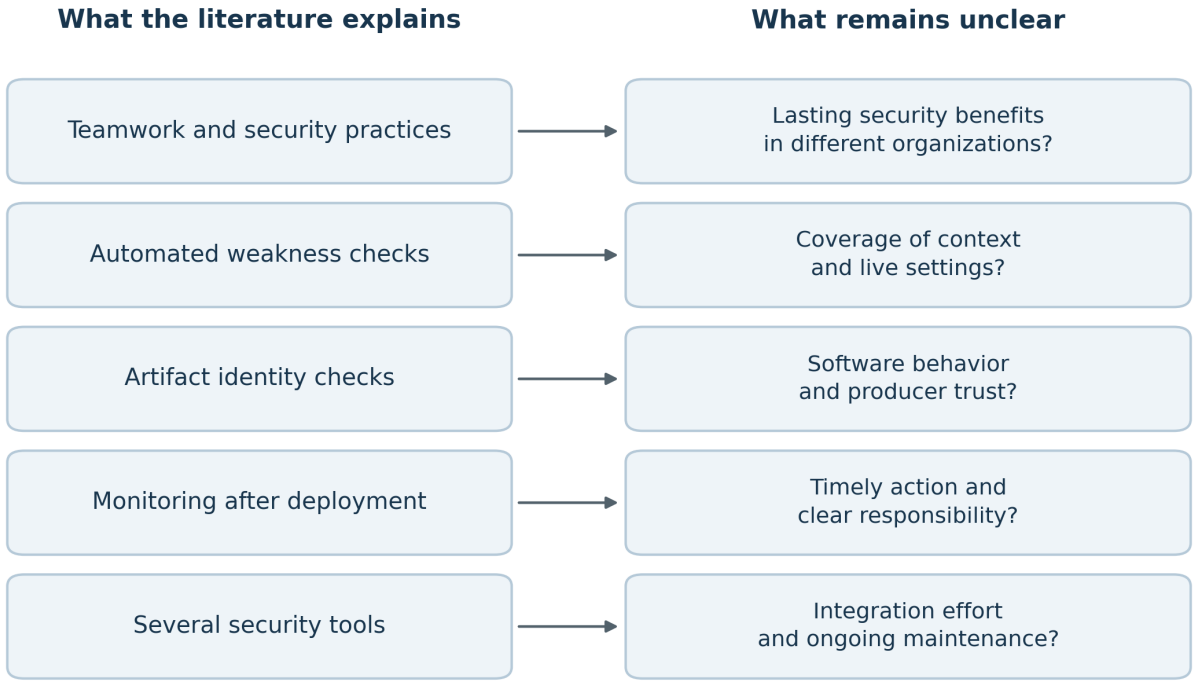
Table 6. Comparison of security approaches across lifecycle stages.

Stage	Existing approaches	Main benefit	Common limitation
Plan	Threat modeling, requirements, assigned responsibilities	Clarifies what needs protection and who is responsible	The analysis can become outdated or disconnected from development
Code	Source analysis, secret detection, code review	Finds selected weaknesses before release	Rules and available context do not cover every problem
Build	Dependency checks, controlled inputs, isolated builds	Examines components and limits exposure during production	Unknown flaws and compromised trusted inputs remain possible
Test	Application testing and requirements-based assessment	Examines behavior and security requirements	Untested paths and different environments limit coverage
Release	Signatures, provenance, artifact verification	Helps confirm artifact identity and expected production	Valid records do not prove harmless content
Deploy	IaC checks, limited identities, configuration policies	Reviews intended settings and deployment permissions	Automation behavior and later changes can affect the actual result
Operate and monitor	Live-state checks, logs, vulnerability monitoring	Helps identify changing conditions after deployment	Missing information or unclear ownership can delay response

6.6 Summary of open questions

Figure 8 and Table 7 bring together the main questions identified in the selected material. These are areas that need further attention within this review. They should not be read as proof that no other researchers have studied them.

Figure 8. Open questions identified through the literature comparison.



Wider literature may address questions that remain open in this review.

The figure summarizes the discussion. It contains no performance measurements or new experimental findings.

Table 7. Research gaps and possible directions for further study.

Topic	What existing work explains	Remaining question	Further work needed
Teamwork and adoption	How teams report security practices and collaboration	How these practices support security over time in different settings	Reviews of more organizations and longer-term experience
Automated detection	How tools identify selected weakness patterns	How context, coverage, and repeated findings affect usefulness	Clearer comparison of tool scope, assumptions, and limitations
Infrastructure behavior	How declarations and automation defects affect resources	How intended settings relate to executed changes and live state	Closer connection between configuration and runtime research
Supply-chain integrity	How signatures and provenance describe production	What verified records establish about behavior and producer trust	Clearer explanations of trust assumptions and receiving-system checks
Identity and secrets	Why credential hygiene and limited access matter	How permissions interact across connected tools and temporary identities	More detailed accounts of access relationships throughout delivery
Monitoring and response	How monitoring reveals conditions after deployment	How findings become timely action with clear ownership	Stronger connections between detection and response studies
Tool integration	How specialized controls fit into delivery stages	How duplication and maintenance affect continued use	More comparison of integration effort and developer experience

7. Discussion and Limitations

7.1 Answers to the research questions

For **RQ1**, the reviewed studies describe DevSecOps as both a team practice and a technical process. Collaboration and clear ownership help teams include security in everyday delivery work. Technical controls examine code, dependencies, infrastructure, released artifacts, and running systems.

For **RQ2**, different approaches support different stages. Source and configuration checks identify selected weaknesses before deployment. Access controls limit permissions. Supply-chain checks help verify expected production and artifact identity. Monitoring examines conditions after deployment.

For **RQ3**, the studies provide findings from specific settings, while frameworks describe risks, requirements, or recommended checks. Table 2 compares the studies' methods and limitations. Table 3 explains why the frameworks serve different purposes and cannot be treated as interchangeable security measures.

For **RQ4**, the main remaining issues are detection context, infrastructure behavior, trust in security records, effective permissions, maintenance effort, and response. These questions come from comparing the selected material. A wider search may identify additional work or different conclusions.

7.2 What the comparison suggests

The reviewed material suggests that security cannot be described simply by whether a pipeline has security tools. Real systems have different controls, exceptions, inherited protections, and operating constraints. Comparing approaches requires explaining the risk each one addresses and the questions it leaves open.

The number of installed tools is also an incomplete measure. Tools may repeat the same checks or rely on the same execution environment. A framework can guide a review without automatically satisfying its requirements. Monitoring can collect information without ensuring that a problem is resolved.

The main lesson is that technical controls and team responsibilities need to work together. Early feedback, limited permissions, artifact verification, configuration checks, and ongoing response address different concerns. Their use should reflect the application, infrastructure, organization, and risks involved.

7.3 Limitations of this review

Source selection: The review uses a selected set of accessible sources rather than a complete systematic search. The five academic studies share a coauthor, so independent perspectives are limited. The discussion cannot represent the whole field.

Source versions: The available manuscripts may differ from final publications, and standards change over time. Some numerical differences in the source versions require caution. This paper therefore emphasizes consistent qualitative findings and identifies the framework versions reviewed.

Different evidence types: Surveys, software artifacts, standards, and incidents have different strengths and limitations. Their findings are discussed together, but are not combined into one performance estimate.

Incident information: The SolarWinds and Codecov sections rely on summaries in the reviewed guidance. These examples help explain possible failures, but do not show their overall frequency or prove that one set of controls would have prevented an entire attack.

Application to other settings: Organizations use different technologies and face different threats. The review does not establish that any approach provides complete protection in every setting.

8. Conclusion

This paper reviewed selected studies and guidance on security integration in CI/CD pipelines. The academic studies describe teamwork, infrastructure weaknesses, automation defects, and configuration-tool limitations. The frameworks provide different ways to examine application requirements, pipeline risks, dependencies, and artifact verification.

The comparison suggests that each approach should be understood within its scope. A reported practice does not prove effective protection, and a warning does not prove that exploitation has occurred. Similarly, an approved configuration may differ from the live environment, and a valid signature does not guarantee harmless software.

The figures and tables summarize these relationships and the questions that remain open. The main conclusion is that DevSecOps requires several supporting practices, clear responsibilities, and continued maintenance across the delivery lifecycle. Broader literature coverage would strengthen the review and include more independent perspectives. The paper provides a focused discussion of existing work, without claiming new experimental results or a new technical solution.